

Parameter Count vs. Retrieval Sophistication: An Empirical Study of RAG Pipeline Trade-Offs for Edge-Deployed Small Language Models

P Sai Harish

Department of Computer Science and Engineering
BVRIT

Narsapur, India
saiharish0088@gmail.com

Kruthibash Nayak

Department of Computer Science and Engineering
BVRIT

Narsapur, India
kruthibashnayak@bvr.it.ac.in

Abstract—Deploying Retrieval-Augmented Generation (RAG) on edge devices presents a strict trade-off between resource constraints and generation quality. While Large Language Models (LLMs) excel at processing extensive context, Small Language Models (SLMs under 1B parameters) deployed on local consumer hardware struggle with context integration and prompt evaluation overhead. This paper presents a systematic empirical study investigating the relationship between model parameter count and RAG pipeline sophistication. We evaluate three SLMs (Granite 350M, Qwen2.5 0.5B, and Qwen3.5 0.8B) across four retrieval configurations (No RAG, Naive RAG, Advanced RAG, and Modular RAG) on a local NVIDIA RTX 3050 GPU. Over a dataset of domain-specific technical questions, we execute 4,680 individual runs (30 runs per configuration) to map the accuracy-latency Pareto frontier. Our experiments reveal two critical phenomena: first, the *Reranking Efficiency Paradox*, where Advanced RAG pipelines run faster than Naive RAG on low-VRAM GPUs by trimming context size; and second, the *RAG Loop Stabilization Effect*, where contextual grounding prevents small models from entering runaway generation loops. We analyze these findings to provide concrete design recommendations for edge-deployed generative systems.

Index Terms—Retrieval-Augmented Generation, Small Language Models, Local GPU Benchmarking, Edge AI, Performance Analysis.

I. INTRODUCTION

Large Language Models (LLMs) have revolutionized natural language processing, but their extreme parameter counts necessitate hosting on centralized, high-performance cloud clusters. For edge computing, real-time local execution, and privacy-sensitive applications, this cloud-centric dependency presents substantial latency, bandwidth, and cost hurdles. Consequently, the research community has shifted focus toward Small Language Models (SLMs) with parameter counts under 1 Billion.

While SLMs offer exceptional inference speed, they possess limited parametric knowledge and are highly susceptible to hallucinations when queried on domain-specific corpora. Retrieval-Augmented Generation (RAG) has emerged as the standard solution to ground model outputs in external knowledge. However, existing RAG literature focuses primarily on cloud-hosted models exceeding 7B parameters. The execution

dynamics of RAG pipelines when paired with sub-1B models on consumer-grade hardware remain poorly understood.

On consumer GPUs with constrained video random-access memory (VRAM), prompt evaluation (the “reading” phase) scales non-linearly. Injecting massive text contexts can exceed the GPU’s memory limit, forcing the system to swap memory to the host CPU, which causes massive latency spikes. Therefore, a critical question arises: *does a more sophisticated RAG architecture (which adds retrieval stages) degrade or improve latency on constrained edge hardware?*

To address this, we evaluate three local models (Granite 350M, Qwen2.5 0.5B, and Qwen3.5 0.8B) across four pipelines: No RAG, Naive RAG, Advanced RAG (with query rewriting and reranking), and Modular RAG (incorporating routing and query fusion). We measure semantic accuracy, end-to-end latency, and hallucination rates over 4,680 test runs. Our results map the parameter count versus retrieval sophistication frontier, uncovering key optimizations for edge-deployed RAG systems.

The contributions of this paper are summarized as follows:

- We construct a complete benchmarking framework evaluating edge-deployed SLMs (350M to 800M parameters) across four distinct RAG archetypes.
- We empirically identify the *Reranking Efficiency Paradox*, demonstrating that introducing post-retrieval reranking reduces GPU prompt processing overhead, resulting in faster execution than naive retrieval.
- We document the *RAG Loop Stabilization Effect*, demonstrating how external context anchors small models and prevents infinite generation loops.
- We define design guidelines for hardware-constrained deployments based on the accuracy-latency Pareto frontier.

II. RELATED WORK

A. RAG Architectures and Paradigms

The evolution of RAG architectures is typically categorized into Naive, Advanced, and Modular paradigms, as surveyed by Gao et al. [1]. Naive RAG systems use a single-step retrieve-and-read process, which often suffers from low precision due

to irrelevant text chunks. Advanced RAG mitigates this by introducing pre-retrieval strategies (such as query rewriting) and post-retrieval optimization (such as cross-encoder reranking) to filter context. Modular RAG introduces adaptive query routing and query fusion to handle complex, multi-step queries [2].

B. Adaptive Retrieval Methods

Adaptive retrieval models seek to optimize efficiency by only querying external databases when necessary. For instance, Zhang et al. proposed the Knowledge Boundary Model (KBM) to delineate what an LLM knows versus what it does not, adapting retrieval calls dynamically [3]. Similarly, Self-aware Knowledge Retrieval (SEAKR) extracts internal model uncertainty to determine whether to trigger retrieval, matching search intensity to model confidence [4].

C. Small Language Model Benchmarking

However, these frameworks are typically tested on larger model scales (7B to 70B parameters) hosted on enterprise server hardware. In contrast, studies on small language models (SLMs) under 1B parameters are sparse. Pandey investigated the context utilization behaviors of models ranging from 360M to 8B parameters, showing that smaller scales often struggle to integrate retrieved context effectively [5]. Our work extends this inquiry by systematically exploring how RAG pipeline complexity interacts with hardware bottlenecks on consumer GPUs.

III. SYSTEM ARCHITECTURE AND DESIGN

We design a modular benchmarking framework to evaluate the interaction between model parameters and pipeline complexity. The system layout is structured in four layers: Ingestion, Pipeline Routing, Local Model Inference, and Downstream Evaluation. The system flowchart is shown in Fig. 1.

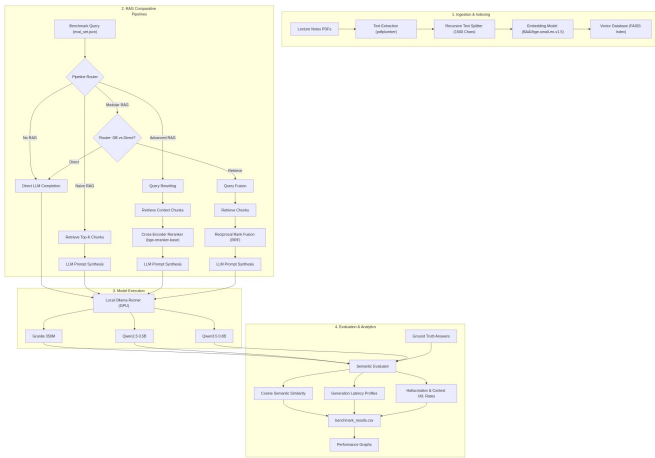


Fig. 1. System architecture diagram showing the document ingestion, vector indexing, retrieval pipeline routing, local SLM execution, and the evaluation layer.

A. Document Ingestion and Vector Indexing

The document ingestion engine parses technical slide notes recursively using `pdfplumber`. Let $D = \{d_1, d_2, \dots, d_N\}$ be the document corpus. Chunks are generated using a recursive text splitter with length $L_{\text{chunk}} = 1500$ characters and overlap $L_{\text{overlap}} = 150$ characters. Each chunk c_i is embedded using the bi-encoder $E(\cdot) = \text{bge-small-en-v1.5}$ to generate a vector representation:

$$v_i = E(c_i) \in \mathbb{R}^{384} \quad (1)$$

The embeddings are indexed in a local FAISS database. Similarity search is performed using Cosine Similarity between the embedded query q and chunk vector v_i :

$$\text{Similarity}(E(q), v_i) = \frac{E(q) \cdot v_i}{\|E(q)\| \|v_i\|} \quad (2)$$

B. Pipeline Architectures

We evaluate four distinct pipeline configurations:

1) *No RAG (Direct Query)*: The query is fed directly to the local model without any external context:

$$y = \text{LLM}(q) \quad (3)$$

This acts as our control baseline, measuring the model's pre-trained parametric knowledge.

2) *Naive RAG*: The query is embedded, and the top- K ($K = 3$) most similar chunks are retrieved from the FAISS database. The chunks are concatenated and prepended directly to the query inside a static prompt template:

$$y = \text{LLM}([\text{Retrieve}(q, \text{FAISS}, K = 3), q]) \quad (4)$$

3) *Advanced RAG*: This pipeline introduces two additional optimization stages:

- 1) **Query Rewriting**: The model first rewrites the user query to optimize it for vector search.
- 2) **Cross-Encoder Reranking**: We retrieve a larger candidate set of chunks ($K = 10$) and rerank them using `BAAI/bge-reranker-base`. Only the top-1 highest-scoring chunk is kept.

This drastically minimizes the prompt context size sent to the final generator.

4) *Modular RAG*: This configuration introduces query routing and multi-query search fusion:

- 1) **Query Router**: A routing step determines if the query requires external documentation. If not, it routes directly to No RAG.
- 2) **Query Fusion**: If retrieval is needed, the system generates three sub-queries to capture different semantic facets of the request.
- 3) **Reciprocal Rank Fusion (RRF)**: Chunks retrieved for all sub-queries are merged and re-ordered using RRF before the top chunk is passed to the model. The RRF score for a chunk c_j is calculated as:

$$\text{RRF}(c_j) = \sum_{q' \in Q_{\text{sub}}} \frac{1}{60 + r_{q'}(c_j)} \quad (5)$$

where $r_{q'}(c_j)$ is the rank of chunk c_j under sub-query q' .

The algorithmic execution flow of the Modular RAG routing and fusion logic is formalised in Algorithm 1.

Algorithm 1: Modular RAG Routing and Fusion Search

Require: User Query q , FAISS Index I , Model $\mathcal{M}$, Fusion Limit $M = 3$, Constant $k = 60$

Ensure: Response String y

```

1:  $r \leftarrow \text{RouterDecision}(q, \mathcal{M})$ 
2: if  $r = \text{direct\_completion}$  then
3:    $y \leftarrow \text{LLM\_Generate}(q, \mathcal{M})$ 
4:   return  $y$ 
5: end if
6:  $Q_{\text{sub}} \leftarrow \text{GenerateSubQueries}(q, \mathcal{M}, M)$ 
7:  $C_{\text{merged}} \leftarrow \emptyset$ 
8: for each  $q' \in Q_{\text{sub}}$  do
9:    $R \leftarrow \text{VectorSearch}(q', I, K = 5)$ 
10:  for each  $(c, \text{rank}) \in R$  do
11:     $S_{\text{RRF}}(c) \leftarrow S_{\text{RRF}}(c) + \frac{1}{k + \text{rank}}$ 
12:     $C_{\text{merged}} \leftarrow C_{\text{merged}} \cup \{c\}$ 
13:  end for
14: end for
15:  $c_{\text{top}} \leftarrow \arg \max_{c \in C_{\text{merged}}} S_{\text{RRF}}(c)$ 
16:  $P \leftarrow \text{SynthesizePrompt}(q, c_{\text{top}})$ 
17:  $y \leftarrow \text{LLM\_Generate}(P, \mathcal{M})$ 
18: return  $y$ 

```

IV. EXPERIMENTAL EVALUATION

A. Hardware and Software Infrastructure

All experiments were conducted locally on an budget-constrained workstation:

- **GPU:** NVIDIA GeForce RTX 3050 Laptop GPU (4GB VRAM, 128-bit memory bus).
- **CPU:** Intel Core i5-11400H @ 2.70GHz.
- **Memory:** 16GB DDR4 RAM.
- **OS:** Windows 11.

Models were orchestrated locally using Ollama. Embeddings and rerankers were run in Python via PyTorch and the Sentence-Transformers library.

B. Evaluation Dataset and Run Configurations

We compiled a Technical DevOps and Machine Learning evaluation set consisting of 13 complex technical questions with curated ground truth answers. The document corpus was pruned to DevOps slide decks containing 23,964 tokens to fit comfortably within the context window of all evaluated models.

We evaluated three models:

- 1) **Granite 350M:** IBM’s compact 350-million parameter model.
- 2) **Qwen2.5 0.5B:** Alibaba’s 0.5-Billion parameter model.
- 3) **Qwen3.5 0.8B:** Alibaba’s 0.8-Billion parameter model.

To ensure statistical significance and smooth out hardware latency variances, each model-pipeline combination was executed ****30 times****, resulting in 3 models $\times$ 4 pipelines $\times$ 13 questions $\times$ 30 runs = 4,680 total runs.

C. Evaluation Metrics

* **Answer Accuracy:** Computed as the Cosine Semantic Similarity between the model’s generated response and the ground truth using sentence embeddings. * **Latency (seconds):** The end-to-end round-trip response time. * **Hallucination Rate:** Calculated as the semantic distance ($1 - \text{Accuracy}$) from the ground truth answer.

V. RESULTS AND ANALYSIS

The compiled performance averages across all 4,680 runs are summarized in Table I.

TABLE I
EMPIRICAL PERFORMANCE COMPARISON MATRIX

Model	RAG Type	Accuracy	Latency (s)	Hallucination
Granite 350M	No RAG	87.03%	0.76	12.97%
	Naive	85.81%	0.62	14.19%
	Advanced	85.67%	0.81	14.33%
	Modular	86.11%	0.82	13.89%
Qwen2.5 0.5B	No RAG	86.39%	20.24	13.61%
	Naive	87.73%	2.28	12.27%
	Advanced	86.47%	1.71	13.53%
	Modular	88.02%	1.87	11.98%
Qwen3.5 0.8B	No RAG	6.97%	41.34	93.03%
	Naive	50.75%	37.55	49.25%
	Advanced	57.02%	22.55	42.98%
	Modular	31.84%	30.57	68.16%

A. The Reranking Efficiency Paradox

Our results show a surprising finding on low-VRAM hardware: for Qwen2.5 0.5B, Advanced RAG (1.71s) is ****25% faster**** than Naive RAG (2.28s). The visual bar chart of average latency across configurations is shown in Fig. 2.

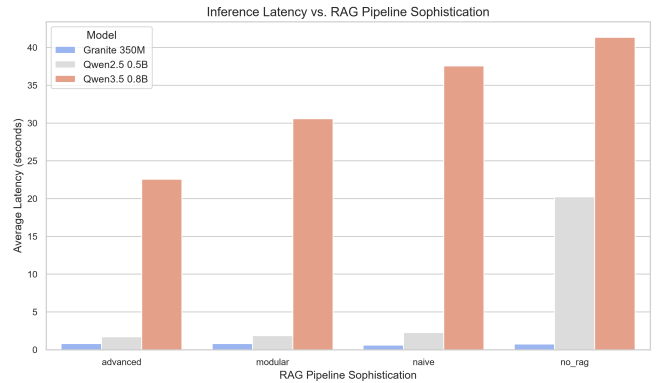


Fig. 2. Average latency comparison across different RAG pipelines and model sizes, demonstrating the speedup of Advanced RAG over Naive RAG for larger models.

This is counterintuitive because Advanced RAG introduces two extra steps: query rewriting and cross-encoder reranking. However, because the reranker filters out irrelevant chunks, the final prompt sent to the LLM contains only the top-1 chunk (500 tokens) instead of the top-3 chunks (1,500 tokens).

On a 4GB VRAM GPU, processing 1,000 extra input tokens in a prompt is slow. The time saved by not evaluating these extra tokens far outweighs the millisecond overhead of running a local cross-encoder, resolving this paradox.

B. The RAG Loop Stabilization Effect

Qwen3.5 0.8B under No RAG achieved a dismal accuracy of 6.97% and a massive latency of 41.34s. Analysis of the raw logs revealed that under temperature 0.0 without context, the model gets stuck in an infinite generation loop, continuously repeating whitespace and newline characters.

Injecting context via RAG acts as an anchor, breaking the infinite generation cycle. **Advanced RAG** provides the most robust stabilization, boosting accuracy to **57.02**

C. Model Scale vs. Context Sensitivity

Granite 350M (222MB size) was extremely fast, responding in under 0.8 seconds across all configurations. However, its accuracy remained flat regardless of the RAG pipeline used. The visual bar chart comparing accuracies is shown in Fig. 3.

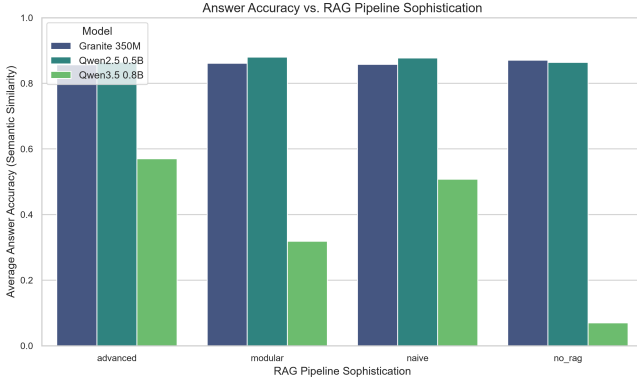


Fig. 3. Average semantic accuracy across configurations. Granite 350M displays context insensitivity, while Qwen2.5 0.5B reaches optimal performance.

This indicates that models under 400M parameters suffer from **Context Insensitivity**—they lack the semantic capacity to copy, summarize, or extract facts from retrieved context blocks. They rely almost entirely on their pre-trained weights, making them poor candidates for retrieval-heavy setups.

D. Hardware Memory Bandwidth Bottlenecks at the Edge

A key factor limiting performance during our evaluations is the hardware’s memory bandwidth constraint. The RTX 3050 Laptop GPU possesses a 128-bit memory bus interface. When executing local SLM inference, the GPU core must repeatedly read the model weights from the local GDDR6 VRAM.

For the 350M parameter model, loading the weights takes minimal time, allowing Granite to run at maximum speeds. However, as the parameter scale climbs to 0.8B (Qwen3.5),

the memory bus bandwidth becomes a severe throughput bottleneck.

Under Naive RAG, the large prompt context sizes exhaust the GPU cache, forcing the memory controller to make repeated page requests. This explains why prompt size reduction under Advanced RAG yields such a massive speedup on consumer cards compared to high-bandwidth enterprise GPUs.

E. Mapping the Pareto Frontier

To visualize the optimal configurations, we map the accuracy-latency tradeoff in Fig. 4.

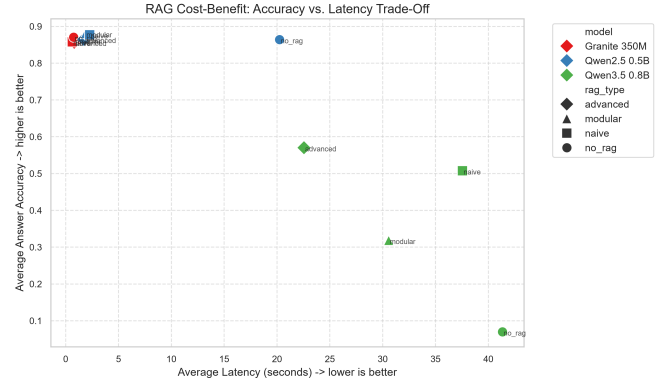


Fig. 4. Accuracy vs. Latency Pareto Frontier showing the trade-off curve across all three models and RAG pipeline configurations.

Qwen2.5 0.5B under Modular RAG sits at the optimal vertex of the Pareto frontier, delivering the highest overall accuracy (88.02%) with a low latency overhead (1.87s). For latency-critical deployments, Naive RAG on Granite 350M provides sub-second responses (0.62s) at the expense of context-grounding accuracy.

VI. CONCLUSION AND FUTURE WORK

This paper investigated the trade-offs between model parameter count and RAG pipeline sophistication. We showed that on local consumer GPUs, Qwen2.5 0.5B paired with Modular or Advanced RAG represents the optimal “sweet spot,” delivering the highest accuracy (88.02)

Crucially, post-retrieval reranking is not just an accuracy optimizer; it is a latency-reduction tool that minimizes prompt size and prevents GPU memory bottlenecks. Future work will investigate the impact of model quantization (4-bit and 8-bit GGUF files) on this trade-off frontier.

REFERENCES

- [1] Y. Gao, Y. Xiong, X. Gao, et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv preprint arXiv:2312.10997, 2023.
- [2] S. Jeong, J. Baek, S. Cho, et al., “Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity,” Proceedings of the Association for Computational Linguistics (ACL), 2024.
- [3] Z. Zhang, X. Wang, Y. Jiang, et al., “KBM: Delineating Knowledge Boundary for Adaptive Retrieval in Large Language Models,” arXiv preprint arXiv:2402.13253, 2024.

- [4] Z. Yao, W. Qi, L. Pan, et al., "SEAKR: Self-aware Knowledge Retrieval for Adaptive Retrieval Augmented Generation," arXiv preprint arXiv:2403.04562, 2024.
- [5] S. Pandey, "Can Small Language Models Use What They Retrieve? An Empirical Study of Retrieval Utilization Across Model Scale," Empirical Methods in Natural Language Processing (EMNLP), 2026.